

Informe TP – Test de carga y optimización del microservicio /extract

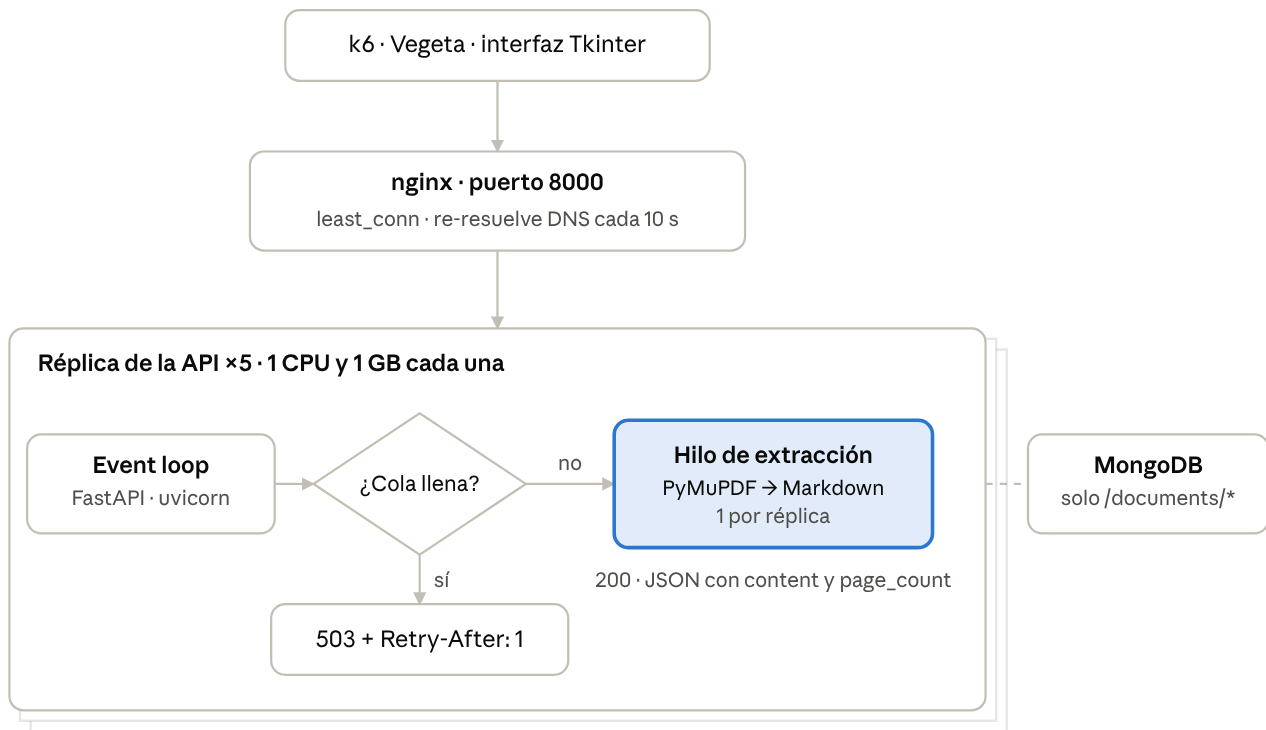
Resumen

Con 5 réplicas limitadas a 1 CPU y 1 GB cada una, el servicio sostiene entre 47 y 50 req/s en el spike de k6 con 0 % de errores, y resuelve el 100 % de la carga fija de Vegeta (50 req/s) con una mediana de 0.12 s. El microservicio de referencia de la cátedra procesa 25.35 req/s en k6 y completa el 66.53 % en Vegeta.

La mejora sale de cuatro cambios, cada uno medido antes de adoptarlo: apagar el OCR por defecto, sacar la extracción del event loop a un worker dedicado, acotar la cola con rechazo rápido (503) y repartir la carga entre 5 réplicas detrás de nginx. La conversión a Markdown y el contrato del endpoint (`POST /extract` → `{content, page_count}`) no cambiaron durante la optimización.

Arquitectura de la solución

`docker compose up --build` levanta 7 contenedores: nginx, 5 réplicas de la API y MongoDB, todos con límite explícito de CPU y memoria.

nginx reparte entre 5 réplicas; cada una rechaza con 503 al llenarse

arquitectura de docker-compose.yml · 7 contenedores

Cada réplica separa el runtime HTTP de la extracción: el event loop recibe el PDF y lo pasa a un hilo dedicado, así que nunca se bloquea extrayendo. Si ya hay 100 extracciones en curso o en espera, responde 503 antes de leer el body. MongoDB solo lo usan los endpoints `/documents/*` de la aplicación original; `/extract` no guarda estado.

Contenedor	Límite de CPU	Límite de memoria
api (×5)	1.0	1 GB
nginx	1.0	256 MB
mongodb	0.5	512 MB

La configuración entra por variables de entorno (`APP_OCR_ENABLED`, `APP_EXTRACT_WORKERS`, `APP_MAX_PENDING_EXTRACTIONS`, `APP_MAX_PDF_SIZE_BYTES`) y los logs salen a stdout, como pide Twelve-Factor.

Cuello de botella identificado

El límite es la CPU de cada réplica: un PDF cuesta en promedio unos 82 ms de CPU, así que 1 core no puede pasar de unas 12 req/s. En el baseline, la CPU de la única réplica estuvo al 88 % durante toda la prueba de Vegeta, mientras la memoria crecía hasta 700 MB.

Etapa de la extracción	Peso en la CPU por PDF
MuPDF interpretando el contenido de cada página (código C)	65-80 %
Conversión del resultado a estructuras de Python (<code>get_text("dict")</code>)	20-30 %
Armado del Markdown (tamaño del cuerpo, títulos, unión)	5-8 %
Abrir el PDF y ordenar los bloques	< 2 %

A ese costo se sumaban dos agravantes de diseño:

- **Cola sin límite.** El servidor aceptaba todas las peticiones y las ejecutaba a la vez en 40 hilos sobre 1 core. Todas avanzaban lento al mismo tiempo, y el 74.5 % terminó en timeout de cliente.
- **OCR siempre activo.** Tesseract corría sobre cada imagen. En el PDF de 8.9 MB llevaba la extracción de 0.19 s a 23.8 s, para sumar un 2.6 % de caracteres, en su mayoría ilegibles.

Proceso de investigación

Cada cambio se midió con los mismos scripts de k6 y Vegeta antes de adoptarlo, y se descartó todo lo que no mejoraba los números. Las mediciones del baseline y de la configuración final están en la Comparativa; acá van las iteraciones intermedias.

#	Qué se probó	k6: req/s · p95	Vegeta: éxito · p50	Qué aprendimos
1	Baseline: 1 réplica, extracción en el threadpool sin límite	ver Comparativa	ver Comparativa	CPU al 88 % y cola sin tope: la mayoría de las peticiones expira

#	Qué se probó	k6: req/s · p95	Vegeta: éxito · p50	Qué aprendimos
2	Microbenchmarks de extracción (sin carga)	—	—	Modo <code>blocks</code> ahorra 14 % de CPU pero pierde los títulos; pypdfium2 no es más rápido; multipart cuesta 23 ms contra 4.5 ms del binario crudo en el PDF de 8.9 MB
3	k6 con binario crudo + pool de procesos + cola de 8 (1 réplica)	5.8 · — (96.8 % de 503)	28.5 % · 0 timeouts	En modelo cerrado, cada 503 se reintenta al instante y los bodies rechazados consumen la CPU del worker
4	Cola de 100 (1 réplica)	14.7 · 6.87 s	37.1 % · 0 timeouts	Encolar con espera acotada (unos 7 s) rinde más que rechazar agresivamente
5	5 réplicas detrás de nginx con <code>least_conn</code>	51.6 · 2.14 s	100 % · 0.12 s	El primer intento dio 502 en todo: nginx resolvía el DNS una sola vez. Se agregó re-resolución cada 10 s
6	Pool de procesos	44.0–46.4 · 2.39–2.59 s	100 % · 0.54–1.60 s	Cada réplica usa solo 82–85 % de su CPU: el worker espera mientras le llega el PDF por el pipe
7	2 procesos por réplica	39.4 · 3.17 s	100 % · 1.34 s	Peor: dos procesos compiten por 1 core
8	Sin access log de uvicorn	44.5 · 2.54 s	100 % · 1.86 s	Sin efecto medible

#	Qué se probó	k6: req/s · p95	Vegeta: éxito · p50	Qué aprendimos
9	Extracción en un hilo dedicado (final, 2 corridas)	ver Comparativa	ver Comparativa	Sin copia entre procesos la CPU por réplica sube a 90–94 %. Adoptada

Comparativa de métricas

La configuración final supera al microservicio de referencia en todas las métricas de las dos pruebas, con las mismas restricciones de hardware. Los rangos son de dos corridas.

Métrica	Profesor	Baseline (1 réplica)	Final (5 réplicas)
k6: peticiones en 40 s	1.037	447	1.888–1.984
k6: throughput	25.35 req/s	9.18 req/s	47.2–49.5 req/s
k6: errores	0 %	0 %	0 %
k6: p50	1.88 s	2.11 s	1.59 s
k6: p90 / p95	7.83 / 8.80 s	22.83 / 24.06 s	2.58–3.06 / 2.98–3.46 s
k6: máximo	13.94 s	33.87 s	4.57–4.81 s
Vegeta: throughput efectivo	16.65 req/s	6.37 req/s	49.8–49.9 req/s
Vegeta: peticiones exitosas	998 / 1.500 (66.53 %)	382 / 1.500 (25.47 %)	1.500 / 1.500 (100 %)
Vegeta: timeouts de cliente	501 (33.40 %)	1.118 (74.5 %)	0
Vegeta: p50	14.89 s	30.0 s (tope del timeout)	0.12 s
Vegeta: p99 / máximo	—	30.0 / 30.0 s	0.83–1.06 / 1.07–1.47 s

En Vegeta el servicio ya no se satura: 50 req/s quedan por debajo de la capacidad de las 5 réplicas, que ronda las 50–60 req/s según el spike de k6.

Decisiones de diseño

Cada decisión se tomó con una medición; la alternativa descartada figura al lado.

Decisión	Alternativa descartada	Por qué
PyMuPDF en modo <code>dict</code> , con títulos por tamaño de fuente	Modo <code>blocks</code> (14 % menos CPU) o <code>pypdfium2</code>	<code>blocks</code> no trae tamaños de fuente y el Markdown quedaría sin títulos; <code>pypdfium2</code> no fue más rápido
OCR apagado por defecto (<code>APP_OCR_ENABLED</code>)	OCR siempre activo	Multiplica hasta por 125 el tiempo del PDF más pesado y aporta texto mayormente ilegible
Extracción en un hilo dedicado por réplica	Pool de procesos	Con 1 CPU no hay paralelismo que ganar, y copiar el PDF al proceso deja 15–18 % de la CPU sin usar. Contra: la cola de k6 sube de 2.4–2.6 s a 3.0–3.5 s en p95
Cola de hasta 100 extracciones por réplica; después, 503 con <code>Retry-After: 1</code>	Cola de 8	Con 8, k6 recibió 96.8 % de 503. Con 100, la espera máxima ronda los 7 s por réplica, lejos del timeout de 30 s
El 503 se decide antes de leer el body	Leer el PDF y después rechazar	No gasta CPU ni memoria en recibir un PDF que se va a descartar

Decisión	Alternativa descartada	Por qué
1 worker por réplica (<code>APP_EXTRACT_WORKERS=1</code>)	2 workers	Con el límite de 1 CPU aplicado, 2 workers bajaron el throughput de 44-46 a 39.4 req/s
nginx con <code>least_conn</code> y re-resolución de DNS cada 10 s	Round-robin; DNS resuelto al arrancar	Los PDFs varían de 35 a 130 ms: <code>least_conn</code> evita apilar los pesados en una réplica. Sin re-resolución, una réplica reiniciada cambia de IP y nginx responde 502
nginx reenvía el body sin guardarlo (<code>proxy_request_buffering off</code>), con límite de 11 MB	Buffering de nginx; límite de 10 MB	El buffering escribe a disco los bodies grandes. Con 10 MB, nginx rechazaba con HTML los PDFs cerca del límite enviados por multipart
k6 envía el PDF como binario crudo	Multipart	El TP permite ambos; el multipart cuesta unos 19 ms extra por petición en el PDF de 8.9 MB. El servidor acepta las dos formas
<code>/extract</code> no usa la base de datos	Reutilizar <code>/documents/upload</code> , que deduplica por checksum en MongoDB	Proceso sin estado (Twelve-Factor) y sin la latencia de Mongo en el camino crítico

Cómo reproducir

Todo se levanta con un solo comando y queda escuchando en el puerto 8000. Los scripts leen los 4 PDFs de `tests/stress/pdfs/` .

1. Levantar el stack (5 réplicas, nginx y MongoDB):

```
docker compose up --build
```

2. Spike con k6 (10 s hasta 100 VUs, 20 s sostenidos, 10 s hasta 0):

```
k6 run tests/stress/spike_tests.js
```

3. Carga fija con Vegeta (50 req/s durante 30 s, timeout de 30 s, rotando los 4 PDFs):

```
bash tests/stress/vegeta_tests.sh
```

Ambos scripts aceptan la variable `BASE_URL` para apuntar a otro host (por defecto `http://localhost:8000`). Para comparar en igualdad, conviene reiniciar el stack entre una prueba y la otra.

Limitaciones y riesgos

- **Medición en el host.** Las pruebas corrieron con Podman 6.1.3 en un host de 16 cores, con k6 y Vegeta en la misma máquina. Con el scheduler `scx_lavd` activo, el límite de CPU no se aplica; las iteraciones 6 a 9 se midieron con el scheduler por defecto. Entre corridas idénticas hubo hasta un 10 % de variación en k6.
- **La cola cuenta peticiones, no bytes.** En el peor caso, 100 PDFs de 10 MB en una sola réplica superarían su límite de 1 GB. En las pruebas no ocurre, porque nginx reparte la carga y k6 tiene 100 VUs en total.
- **Multipart y disco.** Por multipart, Starlette escribe a disco los archivos de más de 1 MB. Los scripts usan binario crudo, que se procesa en memoria.
- **Calidad del Markdown.** Los títulos se detectan por tamaño de fuente: un texto grande que no es título, como el autor en una portada, se marca como `##`.
- **Dependencia de arranque.** Las réplicas esperan a que MongoDB esté sano para arrancar, aunque `/extract` no lo usa.